Technische Universität Dresden

Master Thesis Defense

SBOM Monitor

Parth Dhawan

Supervising Professor: Prof. Dr. Christof Fetzer

Course: Distributed Systems Engineering

Supervisor: Dr. K. Pubudu N. Jayasena

07.01.2026

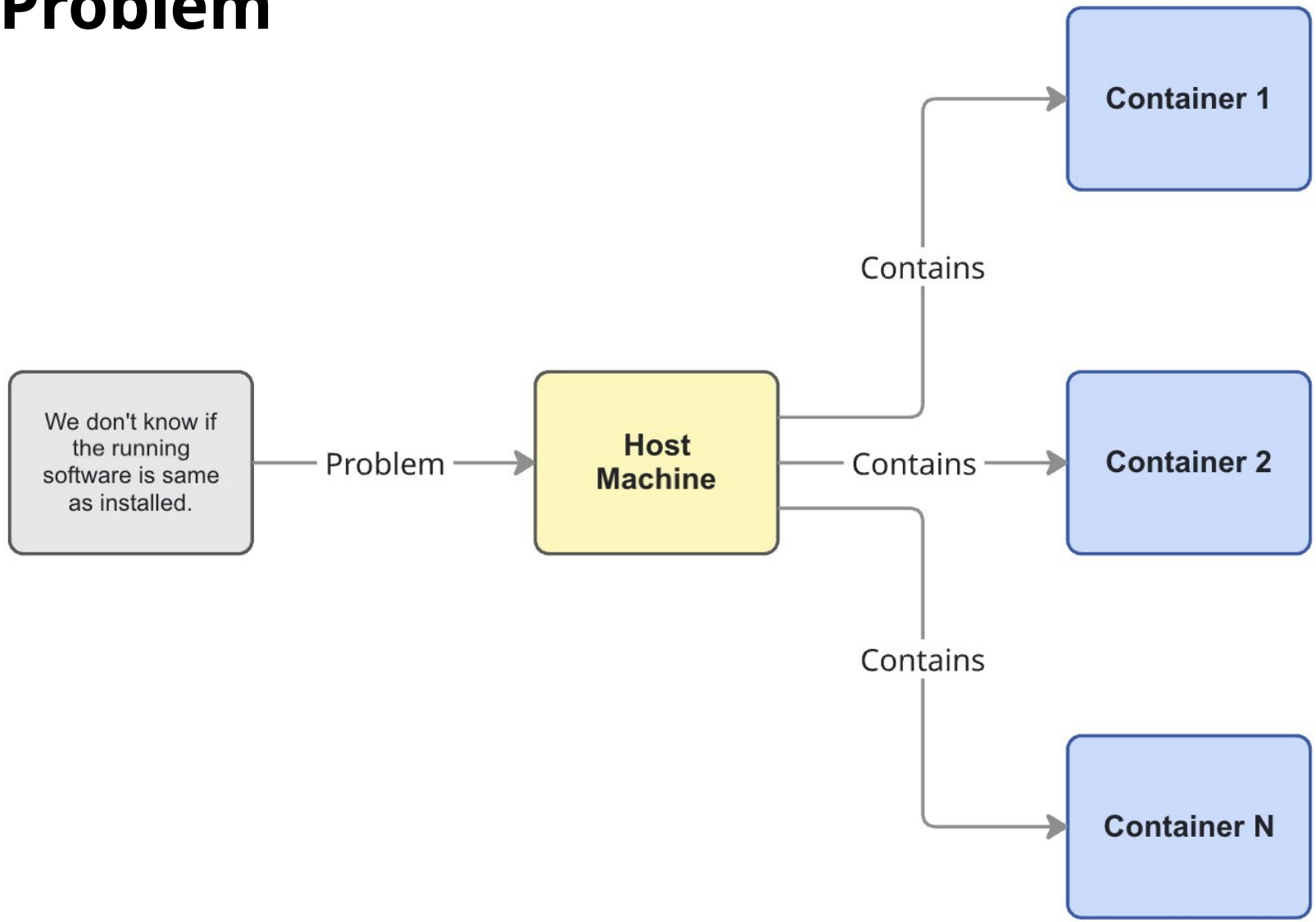
Motivation: The Integrity Gap

Build-Time Assurance (SBOM)	THE GAP 	Runtime Enforcement (IMA)
Declared Software Inventory		*Execution-Time Verification*
• Static record of artifacts • Generated at build	**No Automated Linkage**	• Kernel-level integrity checks • Enforced at execution

Consequence: The disconnect enables supply chain attacks (e.g., SolarWinds, affecting 18,000+ organizations) where compromised build pipelines bypass runtime checks.

Source: U.S. GAO. (2021). SolarWinds Cyberattack Response

The Main Problem



State of the Art: Why Existing Solutions Fall Short?

Approach	Description	Major Limitations
Standard IMA	The default Linux integrity subsystem.	**Lacks Isolation:** Enforces one global policy for the whole machine, cannot distinguish containers from the host.
Container-IMA	Research method to virtualize IMA for containers.	**Non-Standard Architecture:** Relies on custom kernel modifications to track integrity, which are not available in standard upstream Linux.
IMA Namespaces	Emerging technology to create isolated IMA instances.	**Not Production Ready:** Still experimental, requires complex, non-standard modifications to the Linux Kernel.

Our Approach: A monitoring system that provides container visibility today, using only standard, upstream Linux features without kernel patching.

Source: Luo, W., et al. (2019). Container-IMA: A Privacy-Preserving Integrity Measurement Architecture for Containers
Ferro, L. (2023). *Container Attestation with Linux IMA namespaces* (Master's Thesis, Politecnico di Torino)

Research Objectives

- **Designed** a unified framework bridging build-time SBOMs with runtime integrity enforcement.
- **Architected** the solution using standard upstream Linux features, eliminating the need for custom kernel patches.
- **Implemented** automated detection mechanisms to identify unauthorized binaries and file drift.
- **Developed** centralized observability dashboards for near real-time monitoring via Prometheus & Grafana.
- **Validated** system efficacy through rigorous testing in simulated compromise scenarios.

Technische Universität Dresden

Background

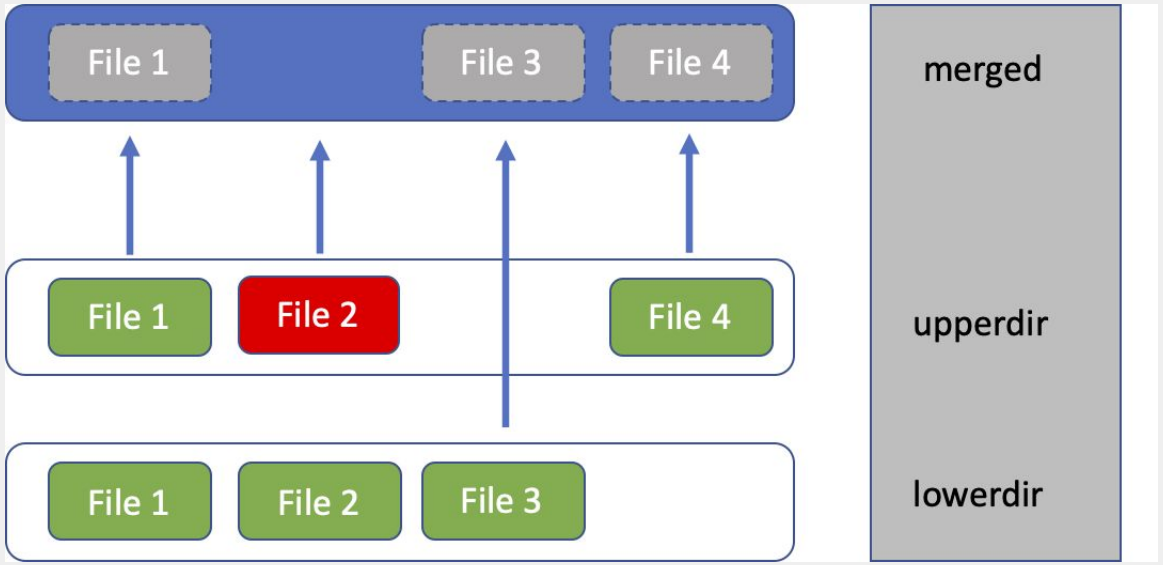
Software Bill of Materials (SBOM)

- **Definition:** A machine-readable inventory of all software components.
- **Standard:** We use **SPDX 2.3** (ISO/IEC 5962) for industry-standard compliance.
- **Key Data:** Tracks Component Names, Versions, and SHA256 Hashes.


```
{
  "name": "grep",
  "SPDXID": "SPDXRef-Package-8cd8a3a0e8242bbe",
  "versionInfo": "3.11-4build1",
  "supplier": "Organization: Ubuntu Developers
 <ubuntu-devel-discuss@lists.ubuntu.com>",
  "downloadLocation": "NONE",
  "filesAnalyzed": false,
  "checksums": [
    {
      "algorithm": "SHA256",
      "checksumValue":
      "275938cc6b1e4ab16f803bcc62961ee88312298d9d36b5b1e7a3590d795e3921"
    }
  ]
```

Container File System Architecture: The OverlayFS Challenge

- **The Structure:** Stacks a **Writable Layer** (Upper) on top of the **Read-Only Lower Layer** (Image).
- **The Risk:** Runtime modifications persist only in the upper layer, often evading static image analysis.
- **The Solution:** Verifies the **Merged View** to inspect the comprehensive filesystem state (Lower + Upper).



Threat Model: Adversary Capabilities and Attack Vectors

Adversary	Capabilities	Attack Vectors	Our Detection Method
Container-Level	File modification in writable layer (OverlayFS).	apt install or package or in Baseline	**Merged View:** Scans combined layers using SBOM + IMA (Periodic Cron checks.
Host-Level	Full system access, ability to modify OS binaries.	apt install or package or in Baseline	**Host IMA:** Enforces signatures on execution + Periodic Cron checks.

Linux Integrity Measurement Architecture (IMA)

- **Measurement:** The kernel calculates the file's hash immediately before execution.
- **Appraisal:** It checks if that hash matches the digital signature stored in the file (xattrs).

  *Note: We use **evmctl** to generate these signatures.*
- **Audit:** If the signature fails, the event is logged to the system audit log.
- **The Limitation:** Standard IMA applies one global policy to the whole system. It cannot distinguish between the Host and specific Containers.

The "Gap" We Are Closing

SBOM Generated
Binaries Signed

Known Good State

apt install, file modification

THE GAP

Scheduled verification detects unauthorized changes

Technische Universität Dresden

Architecture

The Big Idea

The Method:

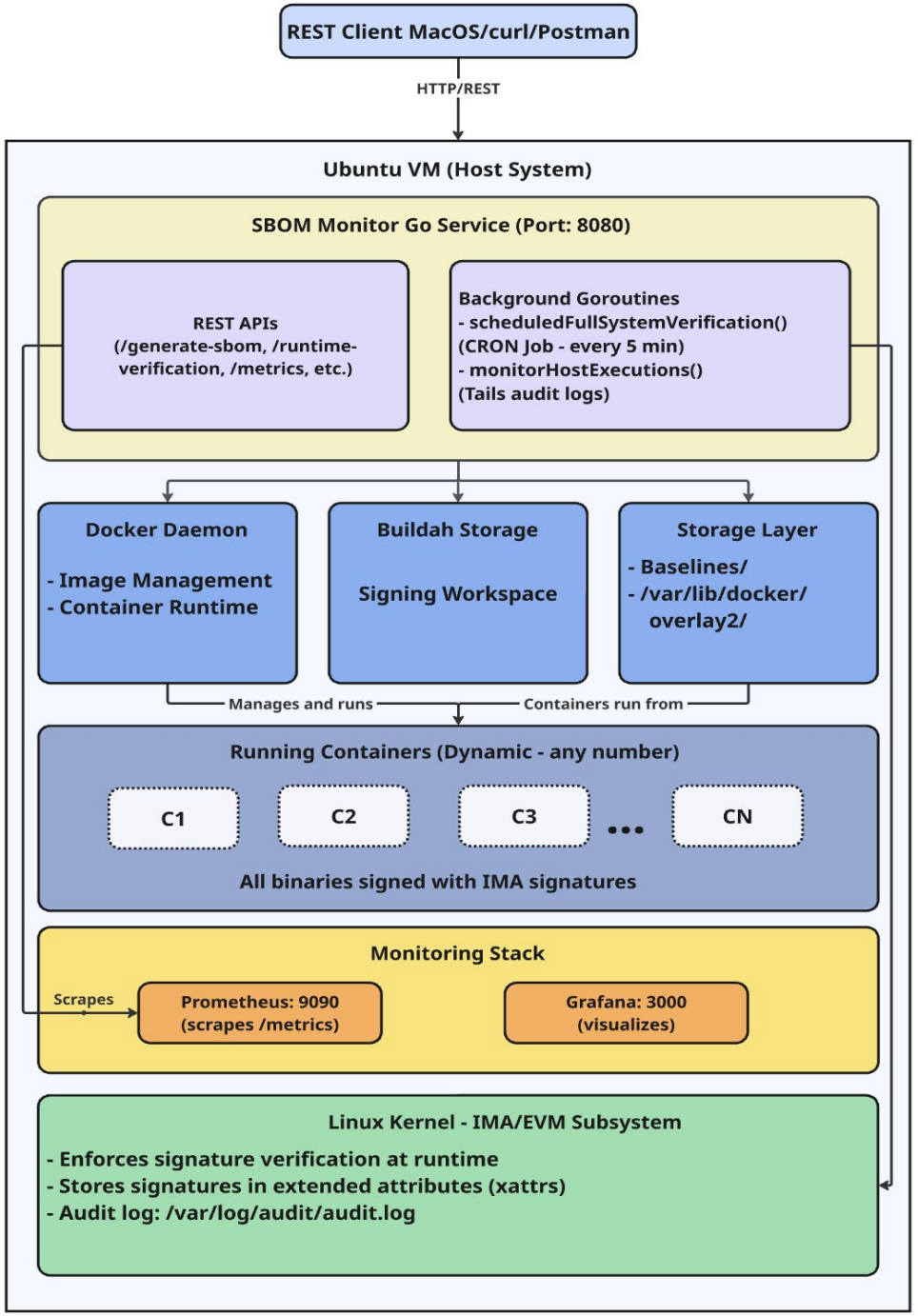
- **Mount OverlayFS:** The Host sees and checks container files directly.
- **Unified Policy:** One standard IMA policy protects everything.
- **No Kernel Patches:** Uses standard, upstream Linux.

Trade-off Analysis:

- **Deployable Today:** Works on standard systems (unlike *IMA Namespaces*).
- **Stable:** Avoids experimental kernel modifications.
- **Requirement:** Depends on runtime tools (Buildah, evmctl) to mount and verify files.

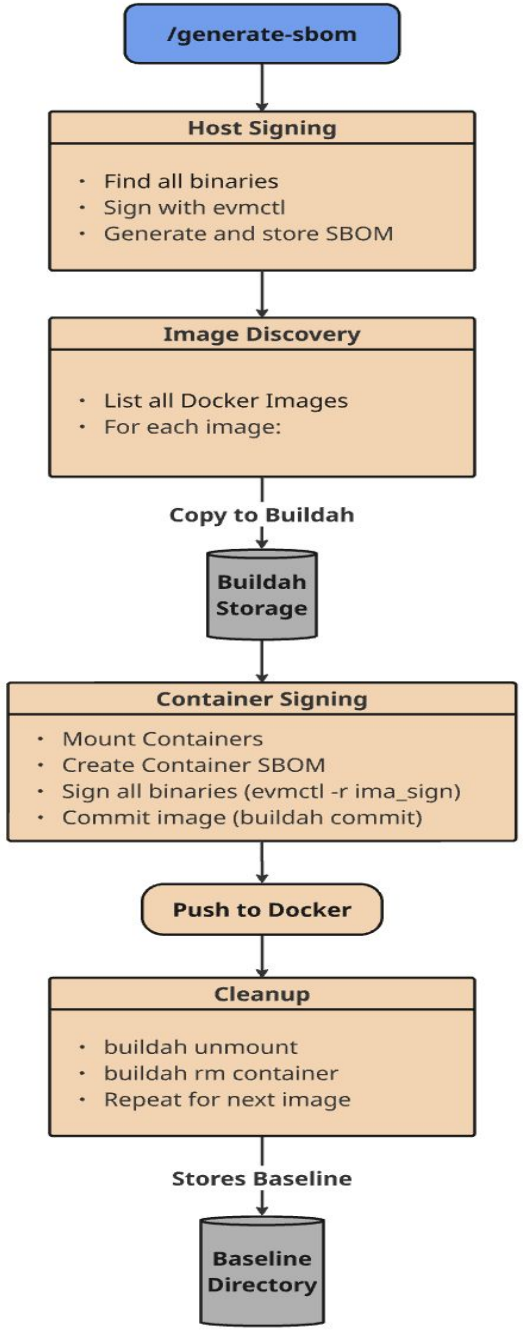
System Components

- **SBOM Monitor Service:** The server of the operation (written in Go).
- **Buildah:** Mounts the container filesystem.
- **Prometheus & Grafana:** Tools to collect data and show graphs.



Signature Workflow

- **Phase 1: Host Signing**
  - **Generate:** Create Host SBOM using Trivy.
  - **Sign:** specific binaries using evmctl (Private Key).
  - **Store:** Save this baseline as the Host "Golden State."
- **Phase 2: Container Signing**
  - **Mount:** Use **Buildah** to expose the container filesystem.
  - **Generate:** Create a container-specific SBOM.
  - **Sign:** Apply evmctl signatures to binaries *inside* the container.
  - **Commit:** Save the signed container image.
- **Result:** A cryptographically secured baseline for both Host and Container.



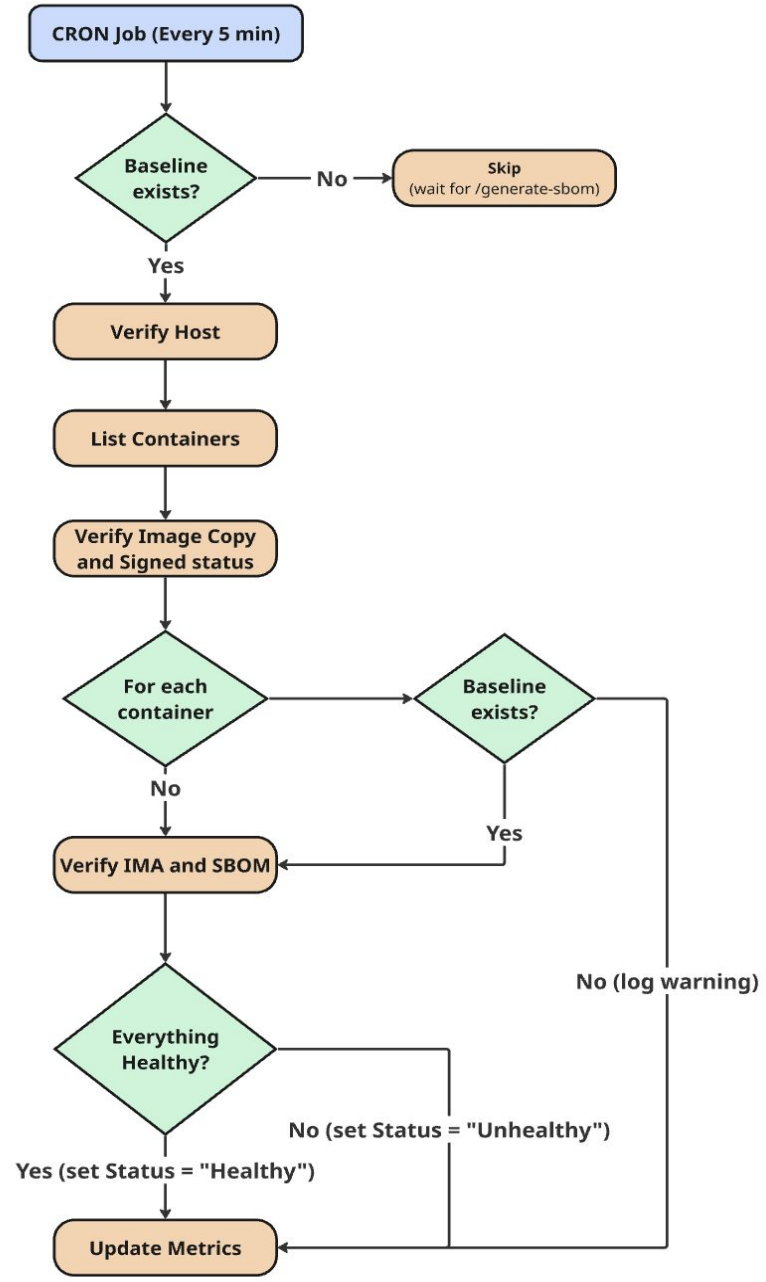
Runtime Verification

1. Scheduled Verification (Proactive)

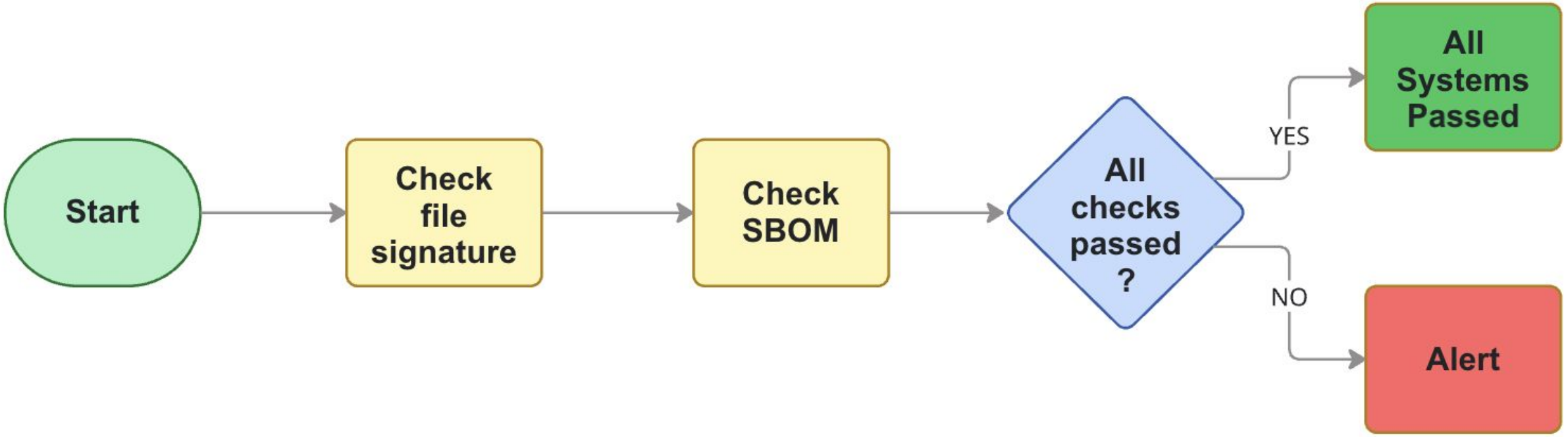
- **Action:** Full filesystem scan every 5 minutes (Cron Job).
- **Goal:** Detects **dormant threats** (files installed but not yet running).

2. Execution Monitoring (Reactive)

- **Action:** Watches the Audit Log (audit.log).
- **Goal:** Instantly alerts if an **unsigned binary executes**.



Verification Decision Logic



Container Verification: Accessing the Merged Filesystem

- **The Structure:** Containers are split into **Lower Layers** (Image) and **Upper Layers** (Runtime Changes).
- **The Command:** We use **Buildah** to instantly expose the full filesystem: buildah mount <containerID>
- **The Solution:** This command returns a path to the **Merged View**. We scan this path to see all files (original + modified) in one place.

Technische Universität Dresden

Implementation

Phase 1: SBOM Generation and Inventory Establishment

- **Tool Selection:** We use **Trivy** for its native support of the **SPDX 2.3** standard (ISO/IEC 5962).
- **Host Scan:** Captures a full inventory of installed packages and their hashes.
- **Container Scan:** Scans image layers to produce isolated, per-container inventories.
- **Output:** A standardized, machine-readable baseline.

Phase 2: IMA Signing

The Mechanism:

- **Algorithm:** We generate **SHA256-RSA** signatures using evmctl.
- **Key:** All binaries are signed using a secure **Private Key**.
- **Storage:** The signature is stored in the binary's extended attribute (security.ima).

The Workflow:

- **Host:** Binaries are signed directly in-place.
- **Containers:** We use **Buildah** to mount the container, sign the binaries inside, and save the image.

Phase 3: Runtime Integrity Verification

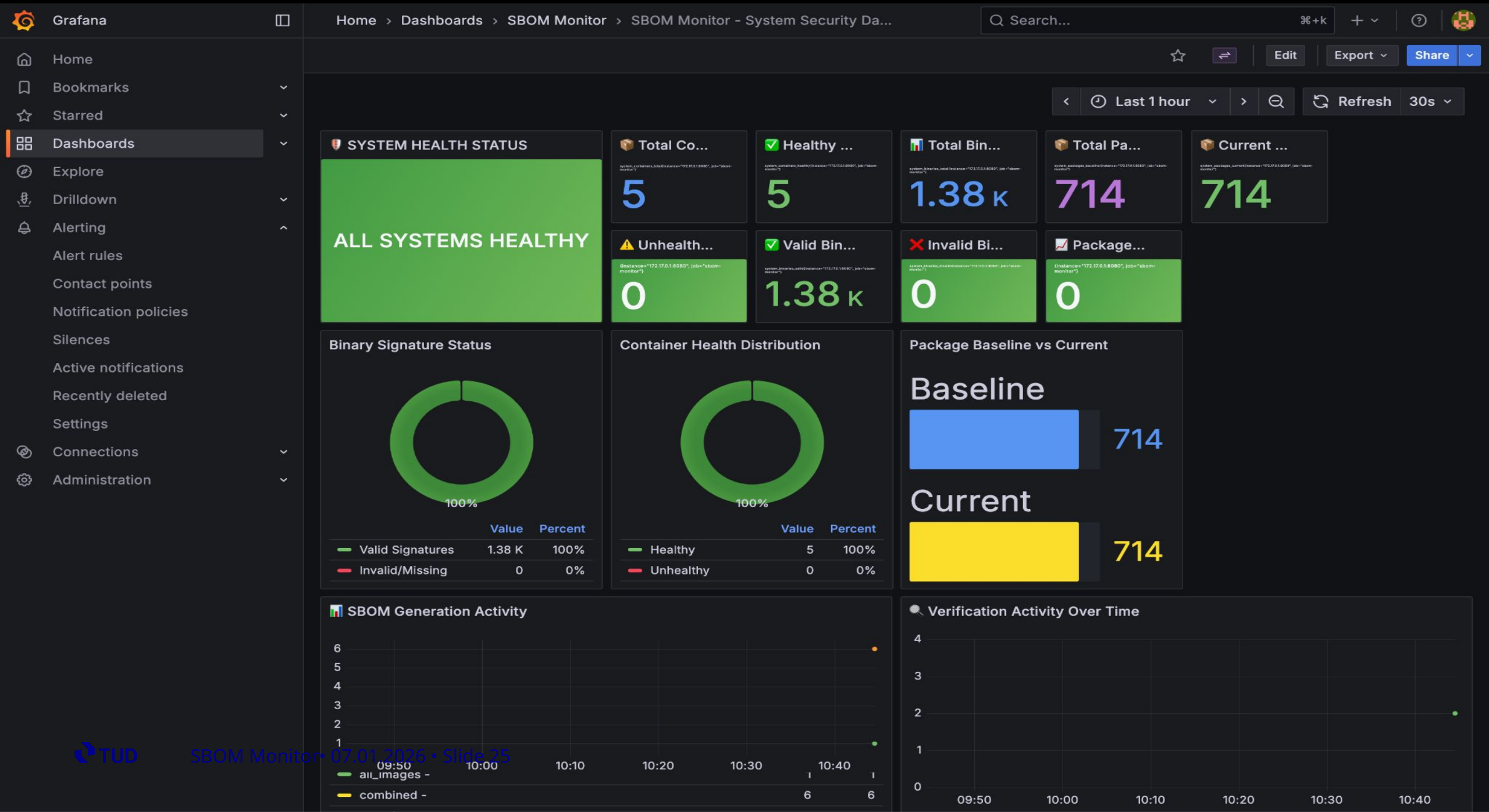
- Executes evmctl verify on targeted binaries to check for tampering.
- Validates the digital signature using the corresponding **Public Key**.
- Compares the binary's current hash against the signature stored in security.ima.
- Triggers an immediate **Failure State** if any unauthorized modification is detected.

Phase 4: Drift Detection

- Detects **Package Drift** when the runtime state diverges from the build-time SBOM.
- Performs periodic re-scans (via Cron) to generate a current inventory of the Host and Containers.
- Compares the live inventory against the trusted "Golden State" baseline established in Phase 1.
- Triggers a **Failure State** if unauthorized packages are added, removed, or modified.

Phase 5: The Dashboard

- Transforms raw audit logs into structured metrics for automated analysis.
- Exposes operational metrics to Prometheus to visualize system status.
- Visualizes system compliance and "Pass/Fail" status via **Grafana** dashboards.


Grafana
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Alert rules
Contact points
Notification policies
Silences
Active notifications
Recently deleted
Settings
Connections
Administration
Home › Dashboards › SBOM Monitor › SBOM Monitor - System Security Da...
Search...
⌘+k
Edit
Export
Share
Last 1 hour
Refresh
30s
SYSTEM HEALTH STATUS
ALL SYSTEMS HEALTHY
Total Co...
5
Healthy ...
5
Total Bin...
1.38 K
Total Pa...
714
Current ...
714
Unhealth...
0
Valid Bin...
1.38 K
Invalid Bi...
0
Package...
0
Binary Signature Status
100%
Value
Percent
Valid Signatures
1.38 K
100%
Invalid/Missing
0
0%
Container Health Distribution
100%
Value
Percent
Healthy
5
100%
Unhealthy
0
0%
Package Baseline vs Current
Baseline
714
Current
714
SBOM Generation Activity
6
5
4
3
2
1
09:50
10:00
10:10
10:20
10:30
10:40
all_images -
combined -
6
6
Verification Activity Over Time
4
3
2
1
0
09:50
10:00
10:10
10:20
10:30
10:40


Grafana
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Alert rules
Contact points
Notification policies
Silences
Active notifications
Recently deleted
Settings
Connections
Administration
Home › Dashboards › SBOM Monitor › SBOM Monitor - System Security Da...
Search...
Edit
Export
Share
Last 1 hour
Refresh
30s
SYSTEM HEALTH STATUS
FAILURES DETECTED
Total Co...
5
Healthy ...
5
Total Bin...
1.38 K
Total Pa...
714
Current ...
723
Unhealth...
0
Valid Bin...
1.38 K
Invalid Bi...
0
Package...
9
Binary Signature Status
100%
Value
Percent
Valid Signatures
1.38 K
100%
Invalid/Missing
0
0%
Container Health Distribution
100%
Value
Percent
Healthy
5
100%
Unhealthy
0
0%
Package Baseline vs Current
Baseline
714
Current
723
SBOM Generation Activity
6
5
4
3
2
1
10:00
10:10
10:20
10:30
10:40
10:50
Name
Last *
Max
all_images -
1
1
Verification Activity Over Time
3
2.75
2.5
2.25
2
1.75
1.5
1.25
1
10:00
10:10
10:20
10:30
10:40
10:50
Name
Last *
Max

Technische Universität Dresden

Validation Results: System Effectiveness Demonstrated

Two Complementary Test Scenarios

Test Environment

- **Host Environment:** Multipass VM Ubuntu version 24.04.03 LTS VM (4 vCPU, 8GB RAM).
- **Container Runtime:** Docker version 28.2.2 integrated with Buildah version 1.33.7
- **Kernel Security:** Linux Kernel IMA enabled.
- **Observability Stack:** Prometheus coupled with Grafana.

Test 1 - Baseline Validation (Healthy State)

Verified the system using the trusted, fully signed baseline.

Outcome: Maintained system integrity with zero deviations observed.

- **2.81K binaries** successfully verified against their signatures.
- **2.08K packages** accurately correlated with the SBOM inventory.
- **Zero alerts** or false positives observed.

SYSTEM HEALTH STATUS

ALL SYSTEMS HEALTHY

Total Co...

system_containers_total{instance="172.17.0.1:8080", job="sbom-monitor"}

4

Healthy ...

system_containers_healthy{instance="172.17.0.1:8080", job="sbom-monitor"}

4

Total Bin...

system_binaries_total{instance="172.17.0.1:8080", job="sbom-monitor"}

2.81 K

Total Pa...

system_packages_baseline{instance="172.17.0.1:8080", job="sbom-monitor"}

2.08 K

Current ...

system_packages_current{instance="172.17.0.1:8080", job="sbom-monitor"}

2.08 K

Unhealth...

{instance="172.17.0.1:8080", job="sbom-monitor"}

0

Valid Bin...

system_binaries_valid{instance="172.17.0.1:8080", job="sbom-monitor"}

2.81 K

Invalid Bi...

system_binaries_invalid{instance="172.17.0.1:8080", job="sbom-monitor"}

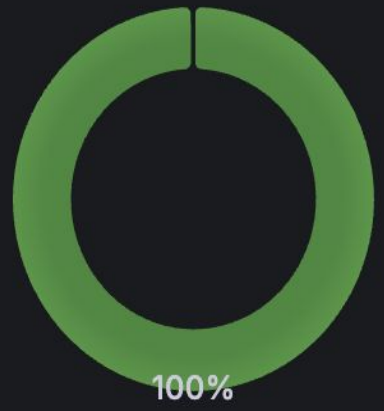
0

Package...

{instance="172.17.0.1:8080", job="sbom-monitor"}

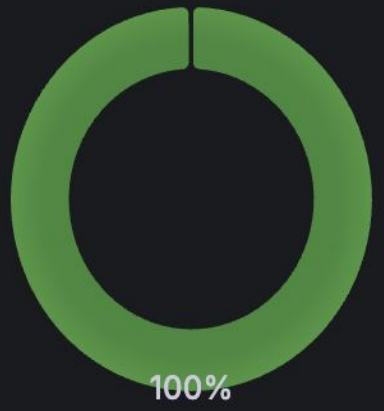
0

Binary Signature Status



	Value	Percent
Valid Signatures	2.81 K	100%
Invalid/Missing	0	0%

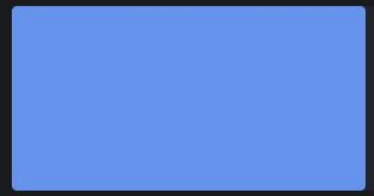
Container Health Distribution



	Value	Percent
Healthy	4	100%
Unhealthy	0	0%

Package Baseline vs Current

Baseline



2.08 K

Current

2.08 K

Test 2 - Attack Simulation (Compromised State)

Injected unauthorized, unsigned programs into the runtime.

Outcome: System achieved **immediate detection** of all unauthorized files.

- **Identified 16 invalid binaries** across **4 containers**.
- **Visual Alert:** Dashboard state shifted to **Red** (Critical).
- System status correctly transitioned to "**Failure State**" confirming the breach.

SYSTEM HEALTH STATUS

FAILURES DETECTED

Total Co...

system_containers_total{instance="172.17.0.1:8080", job="sbom-monitor"}

4

Healthy ...

system_containers_healthy{instance="172.17.0.1:8080", job="sbom-monitor"}

0

Total Bin...

system_binaries_total{instance="172.17.0.1:8080", job="sbom-monitor"}

2.83 K

Total Pa...

system_packages_baseline{instance="172.17.0.1:8080", job="sbom-monitor"}

2.08 K

Current ...

system_packages_current{instance="172.17.0.1:8080", job="sbom-monitor"}

2.08 K

Unhealth...

{instance="172.17.0.1:8080", job="sbom-monitor"}

4

Valid Bin...

system_binaries_valid{instance="172.17.0.1:8080", job="sbom-monitor"}

2.81 K

Invalid Bi...

system_binaries_invalid{instance="172.17.0.1:8080", job="sbom-monitor"}

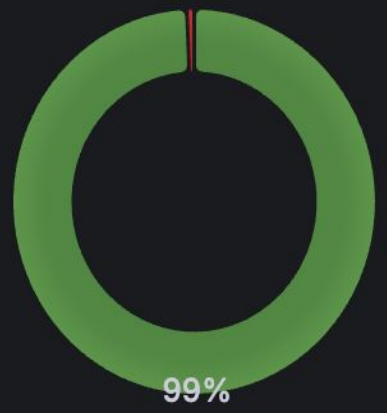
16

Package...

{instance="172.17.0.1:8080", job="sbom-monitor"}

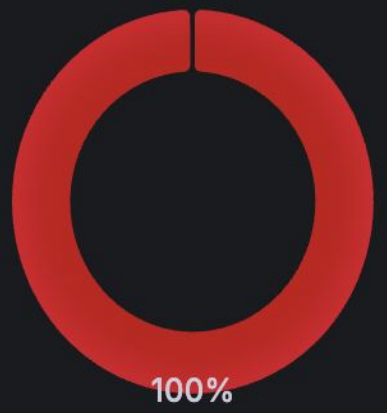
0

Binary Signature Status



	Value	Percent
Valid Signatures	2.81 K	99%
Invalid/Missing	16	1%

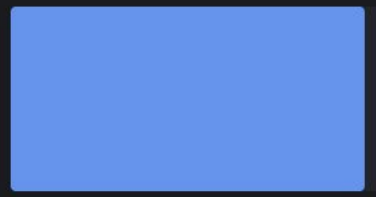
Container Health Distribution



	Value	Percent
Unhealthy	4	100%
Healthy	0	0%

Package Baseline vs Current

Baseline



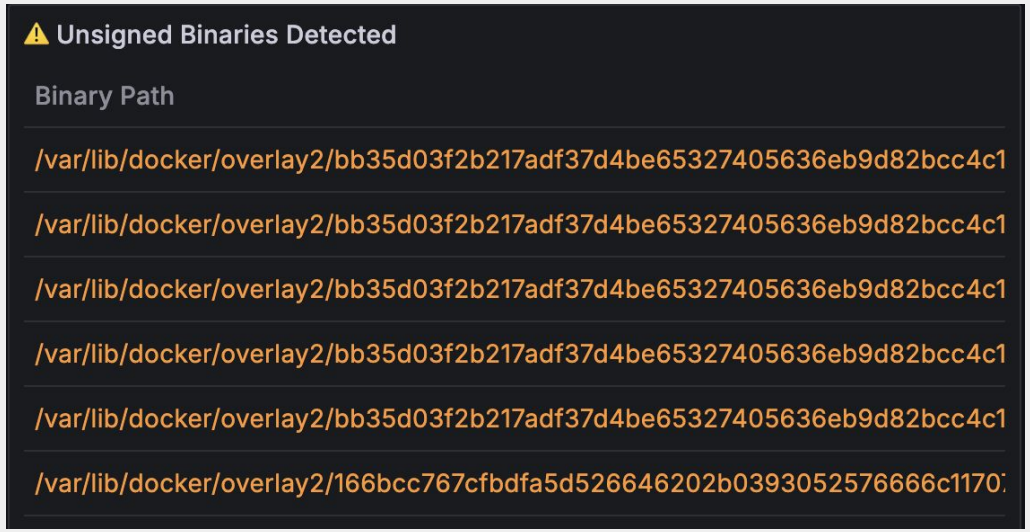
2.08 K

Current

2.08 K

Detected Unsigned Binaries

- Flags exact unauthorized files.
- Lists specific paths for immediate action.



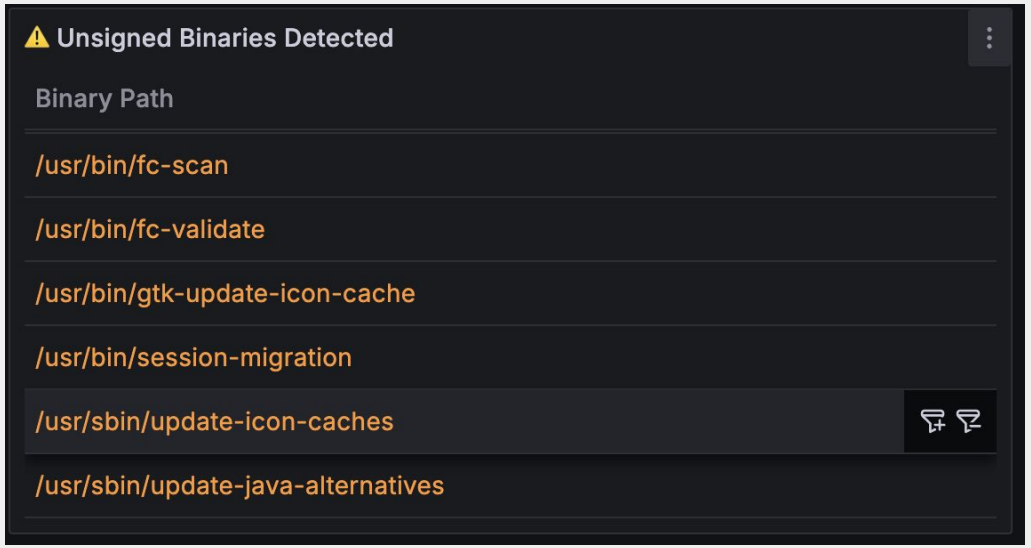
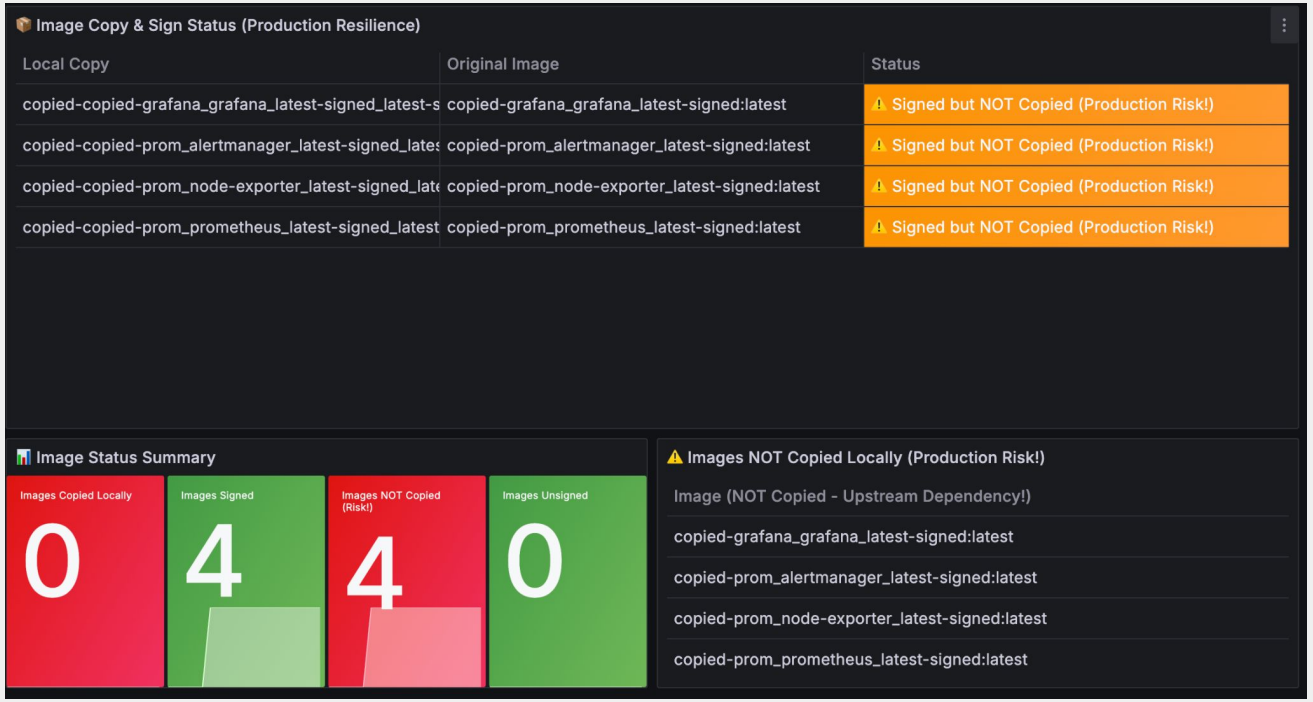
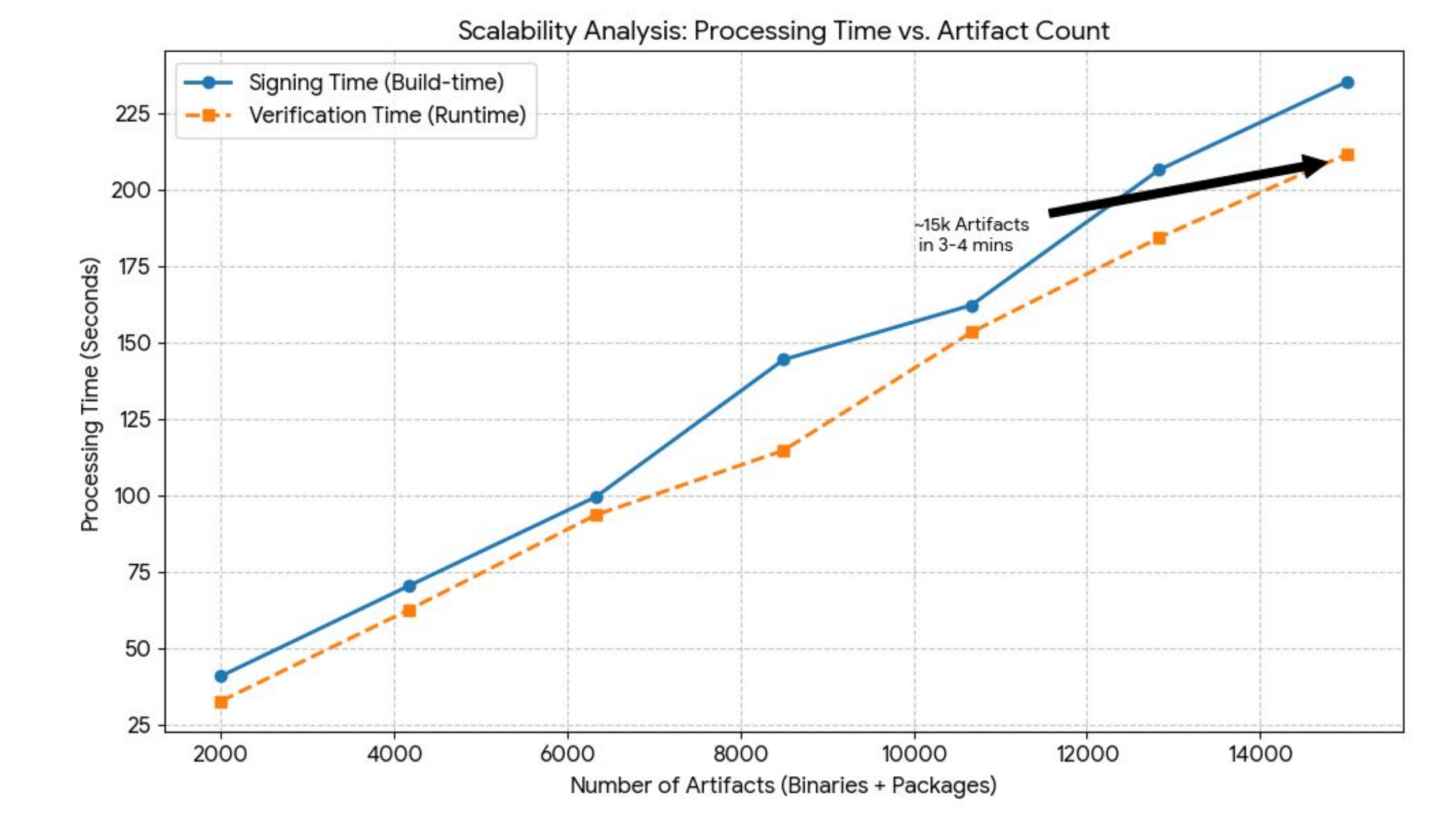


Image Availability & Resilience Monitoring

- Maintains local image replicas to mitigate upstream deletion or registry downtime.
- Validates the presence of offline backups for all active containers via the monitor.
- Flags images lacking local redundancy as a **"Production Risk"** to ensure business continuity.




Scalability Analysis: Processing Time vs. Artifact Count
Signing Time (Build-time)
Verification Time (Runtime)
~15k Artifacts
in 3-4 mins
Processing Time (Seconds)
25
50
75
100
125
150
175
200
225
2000
4000
6000
8000
10000
12000
14000
Number of Artifacts (Binaries + Packages)

Validation Summary: Objectives Achieved

Project Objective	Validation Metric	Key Result	Status
Integrity Verification	Detection Accuracy	In controlled test scenario, detected all the signature mismatches and unsigned binaries.	**Achieved**
SBOM Integration	Baseline Fidelity	Accurate correlation of build-time artifacts to runtime state.	**Achieved**
Observability	Monitoring Solution	Visualizes system integrity and deviation status via Grafana.	**Achieved**
Compatibility	Architecture Standard	Validated using standard upstream Linux (No custom kernel).	**Achieved**

Technische Universität Dresden

Conclusion and Future Work

Contributions and Impact

- **Key Achievements:**
  - **SBOM-to-Runtime Linkage:** Successfully bridged build-time artifacts with runtime enforcement.
  - **Unified Verification:** Secured both Host and Containers without custom kernel modifications.
  - **Operational Observability:** Delivered dashboards for monitoring.

- **Broader Impact:**
  - Enables practical supply chain integrity verification suitable for **regulated industries**.

Future Work

- **Scaling:** Expand support from single VMs to distributed clusters (e.g., Kubernetes).
- **Remote Attestation:** Anchor trust to external services (e.g., SCONE Signer) for stronger security.
- **Rolling Updates:** Allow seamless software updates using Versioned SBOMs.

Technische Universität Dresden

Q/A

Technische Universität Dresden

Appendix

Why SPDX?

- Widespread industry adoption standard: ISO/IEC 5962:2021, used by Linux Foundation.
- It also stores the actual cryptographic hash (SHA256) of the files.

What was the Solarwinds attack?

- **Attack Vector:** Malicious code was injected during the build process ("Sunspot" malware), bypassing source code reviews.
- **Signature Bypass:** The compromised binary was cryptographically signed, rendering standard signature-based verification ineffective.
- **The Verification Gap:** The lack of a **Source-to-Runtime SBOM** comparison meant there was no mechanism to validate that the final binary actually matched the original source code.

Proposal - Signed SBOMs

- Would prevent "Metadata Tampering" by cryptographically signing the SBOM file.
- Can utilize standard tools (e.g., Cosign) to bind the SBOM to a trusted publisher.
- Enables mandatory signature verification at startup before parsing any allowlist data.

The Kernel Patch Problem

- **Single Root of Trust:** Standard Linux architecture utilizes a centralized IMA log and a single TPM for the entire host and all running workloads.
- **Measurement Entanglement:** Artifact hashes from distinct containers are interleaved into one global list, making it impossible to verify a specific container in isolation.
- **High-Cost Isolation:** Solving this via virtualization (e.g., Container-IMA) requires complex kernel patches to simulate private logs, creating fragile dependencies.

Container-IMA

- Relies on invasive kernel patches to simulate virtual TPMs (cPCRs) for containers.
- Incompatible with standard Linux distributions, making it unmaintainable in managed cloud environments.

IMA-Namespace

- Attempts to architecturally isolate integrity policies and measurement logs for individual containers.
- Remains an experimental feature not yet merged into the stable Linux mainline, posing significant stability risks.

Why Mounted Containers instead of Kernel Namespace?

- Tradeoff for deployability.
- IMA Namespaces uses kernel patches, which are not upstream.
- This standard approach can verify both host and container consistently.

Why both Buidah and Docker?

- Buildah has a special feature called buildah mount. It lets you mount the container's hard drive to a folder on your host without running it.
- Docker is the industry standard for running containers.

Handling Updates During Verification

- **Concurrency Control:** Implements **file locking** on storage layers during critical verification windows.
- **OverlayFS Behavior:** Leverages Docker's **Copy-on-Write (CoW)** mechanism. Modifications create new file instances in upper layers.
- **Verification Logic:** Verifier inspects the **Merged View**, ensuring validation of the exact state visible to the application.

Resource Efficiency

- **Minimal Footprint:** Baselines store only **Metadata & Hashes** (JSON), not binary content.
- **Zero-Copy Signatures:** IMA signatures reside in **Extended Attributes (xattrs)**, consuming negligible disk space.
- **Deduplication:** Relies on Docker's native layer sharing, no duplicate storage of container images required.

Does this actually solve TOCTOU (Time-of-Check Time-of-Use) attacks?

Bridging the security gap:

- Re-verifies the entire system every 5 minutes.
- Monitors audit logs to verify binaries immediately upon use.
- Drastically reduces the time window for file swapping attacks.

How softwares updates like apt upgrade are handled?

- Currently, it will be reported as a Package Drift.
- Future works can implement versioned SBOMs.
- Thus, updating the baseline for each “apt upgrade”.

How does this scale to a cluster?

- Currently, it runs inside a single VM.
- Future work can run one agent per node.
- A centralized Prometheus/Grafana instance will be needed to collect metrics from all nodes.

What if an attacker gains Root access on the Host?

- Threat model assumes the Kernel and signing keys are trusted.
- If attacker has root access to Host or hardware TPM, then they can bypass almost any software-based protection.

Remote Attestation in Future Work?

- Currently, verification happens at local machine.
- If local machine is compromised, verification reports can be falsified.
- Tools like SCONE Signer cryptographically binds local integrity reports to a remotely verifiable trust anchor.

Why ima_appraise=log, not enforce?

- **Log Mode (Dev):** Safer for initial setup. It records signature failures without crashing applications.
- **Enforce Mode (Prod):** The end goal. Once the baseline is stable, we switch to strict enforcement to actively block unsigned binaries.

Why files such as /etc/hostname or /.dockerenv are excluded?

- These are dynamically created by container runtime.
- Cannot be signed in advance, because they don’t exist yet.
- Non-executables, so risk of being used for code execution is lower.